
Parameter-Efficient Fine-Tuning of Large Language Models with Low-Rank Adaptation

Jiawei Guo*

Carnegie Mellon University
Pittsburgh, PA, USA

Gen Li†

Carnegie Mellon University
Pittsburgh, PA, USA

Abstract

This report investigates the effectiveness of Low-Rank Adaptation (LoRA) as a parameter-efficient fine-tuning technique for adapting large language models to downstream tasks. We compare LoRA to traditional full fine-tuning, evaluating both computational efficiency and performance. Our experiments demonstrate that LoRA significantly reduces GPU memory consumption and achieves comparable or better performance to full fine-tuning, offering a promising approach for adapting billion-parameter models on limited hardware.

1 Introduction

The field of deep learning is undergoing a remarkable transformation, characterized by the proliferation of increasingly large language models (LLMs) that have demonstrated unprecedented capabilities across a wide spectrum of natural language processing tasks. From early models with millions of parameters to the current generation boasting hundreds of billions, the scale of these architectures continues to grow exponentially. This expansion has been accompanied by a corresponding increase in the complexity and diversity of downstream applications, ranging from intricate question answering and nuanced text generation to sophisticated code synthesis and multi-lingual translation. Adapting a single, powerful pre-trained LLM to this multitude of specific tasks necessitates efficient and adaptable fine-tuning strategies.

Traditional fine-tuning, which involves updating all the parameters of the original pre-trained model, presents significant computational hurdles. For billion-parameter models like BLOOM-1b7, this process demands substantial computational resources, often requiring access to multiple high-end GPUs and extensive training times. Furthermore, storing fully fine-tuned models for each downstream application becomes impractical due to their large memory footprints.

To mitigate these challenges, a variety of parameter-efficient fine-tuning techniques have been proposed. Methods such as adapter modules [6] introduce small, task-specific neural network layers into the pre-trained architecture, allowing for adaptation with a fraction of the original parameters. However, the introduction of these additional layers inevitably increases the model’s depth and computational graph, leading to increased inference latency – a critical concern for latency-sensitive applications. Similarly, techniques like prefix-tuning [9] and prompt-tuning [8] focus on optimizing continuous prompts or prefixes prepended to the input sequence. While parameter-efficient, the optimization landscape for these continuous prompts can be challenging, often exhibiting non-monotonic performance changes and lacking interpretability.

Low-Rank Adaptation (LoRA) [7] has emerged as a compelling alternative, offering a unique balance between parameter efficiency, model quality, and inference efficiency. LoRA freezes the weights of the pre-trained model and introduces a small number of new trainable parameters in the form of

*jiaweigu@andrew.cmu.edu

†genli@andrew.cmu.edu

low-rank decomposition matrices applied to specific weight matrices within the network. By learning these low-rank updates, LoRA significantly reduces the number of trainable parameters, making the adaptation of billion-parameter models like BLOOM-1b7 computationally feasible on more accessible hardware. A key advantage of LoRA is that, unlike adapter-based methods, it does not add any layers to the original model architecture during inference, thus preserving the original inference latency. Furthermore, LoRA has demonstrated the ability to achieve performance comparable to full fine-tuning in many scenarios.

This project investigates the application and potential enhancements of LoRA for adapting the BLOOM-1b7 large language model to multiple downstream tasks. By addressing the limitations of existing parameter-efficient fine-tuning techniques, this research contributes to the development of more practical and scalable approaches for leveraging the power of large language models in real-world applications.

2 Related Work

The increasing scale of pre-trained language models has spurred significant research into parameter-efficient fine-tuning techniques, aiming to adapt these powerful models to downstream tasks while minimizing computational overhead. This section provides a more detailed overview of prominent approaches in this area, including adapter modules, prefix-tuning, prompt-tuning, and low-rank adaptation, culminating in a discussion of the foundational theory and related advancements.

2.1 Adapter Modules

Introduced by Houlsby et al. [6], adapter modules involve inserting small neural network layers (typically with a bottleneck architecture) within the layers of a pre-trained model. During fine-tuning, only the parameters of these adapter modules are updated, while the original model weights remain frozen. This approach significantly reduces the number of trainable parameters and allows for task-specific adaptation. Pfeiffer et al. [11] further extended the use of adapters for multilingual transfer learning. While adapters offer substantial parameter efficiency, a key limitation is the introduction of additional computations during inference, potentially increasing latency. Strategies like adapter pruning and multi-task learning have been explored to mitigate this latency, but the fundamental issue of added layers persists.

2.2 Prefix-Tuning and Prompt-Tuning

Prefix-tuning [9] and prompt-tuning [8] represent alternative parameter-efficient methods that focus on optimizing the input to the pre-trained model rather than modifying the model’s internal parameters. Prefix-tuning involves learning a continuous, task-specific vector (the prefix) that is prepended to the input sequence. Only the parameters of this prefix are trainable. Similarly, prompt-tuning learns continuous prompts that guide the model’s generation. While these methods can be highly parameter-efficient, optimizing the continuous prompt space can be challenging, often leading to non-intuitive and difficult-to-interpret learned prompts. Furthermore, the performance can be sensitive to the initialization and length of the prompt.

2.3 Low-Rank Adaptation (LoRA)

Low-Rank Adaptation (LoRA) [7] offers a distinct approach by leveraging the principle of low-rank approximation [4]. The Eckart-Young-Mirsky theorem states that a matrix can be optimally approximated by another matrix of lower rank. LoRA applies this principle to the weight updates during fine-tuning. Instead of directly updating the full-rank weight matrices of the pre-trained model, LoRA introduces low-rank matrices that represent the task-specific changes. Specifically, for a weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the weight update ΔW is constrained to be of low rank r , such that $\Delta W = BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, with $r \ll \min(d, k)$. During training, only the parameters of the low-rank matrices A and B are learned, while the original weight matrix W_0 is frozen.

The effectiveness of LoRA is further supported by the observation that task-specific updates often reside in a low-dimensional subspace [1]. This suggests that capturing these updates with low-rank

matrices can be sufficient to achieve performance comparable to full fine-tuning with significantly fewer trainable parameters.

LoRA offers several key advantages. Firstly, by freezing the original model weights, multiple small LoRA modules can be trained and stored for different downstream tasks, with the base pre-trained model being shared. Task switching can then be performed efficiently by simply loading the appropriate LoRA modules. Secondly, the reduced number of trainable parameters significantly lowers the computational cost and memory requirements for training, making it feasible to fine-tune very large models on more accessible hardware. Thirdly, since LoRA does not modify the original model architecture during inference, it does not introduce any additional inference latency. Finally, LoRA can be effectively combined with other parameter-efficient techniques like prefix-tuning, offering the potential for further performance improvements.

2.4 Other Parameter-Efficient Techniques

Beyond the methods discussed above, several other parameter-efficient fine-tuning strategies have been explored. AdapterFusion [10] investigates methods for combining knowledge learned by different adapter modules. BitFit [17] proposes fine-tuning only the bias terms of the pre-trained model. Diff-Pruning [5] focuses on identifying and pruning unimportant weights while allowing the remaining weights to be updated. QLoRA [3] further enhances parameter efficiency by quantizing the pre-trained model weights and fine-tuning low-rank adapters.

2.5 Large-Scale Language Models

The development and increasing availability of large-scale language models such as BLOOM [14], T5 [13], and GPT-3 [2] serve as a primary motivation for research in parameter-efficient fine-tuning. The sheer size of these models necessitates efficient adaptation techniques to make them practically usable across a wide range of applications. Our work specifically focuses on adapting the BLOOM-1b7 model, highlighting the relevance of LoRA in the context of such large-scale architectures.

This overview of related work demonstrates the active and diverse research landscape in parameter-efficient fine-tuning. Our project builds upon the promising results of Low-Rank Adaptation and aims to further explore its capabilities and potential for enhancing the adaptability of large language models.

3 Method

Given a pre-trained language model $P_\Phi(y|x)$, where both x and y are sequences of tokens, and Φ represents its parameters. To adapt this model to a specific downstream task, we utilize a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1, \dots, N}$. During conventional full fine-tuning, the model is initialized with the pre-trained weights Φ_0 and these weights are updated by $\Delta\Phi$ through gradient descent to maximize the conditional objective:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{D}} \sum_{t=1}^{|y|} \log(P_\Phi(y_t|x, y_{<t})). \quad (1)$$

A key characteristic of full fine-tuning is that for each distinct downstream application, the model learns a unique set of parameter updates, $\Delta\Phi$.

Low-Rank Adaptation (LoRA) offers a more parameter-efficient alternative by parameterizing the update $\Delta\Phi$ using a significantly smaller set of trainable parameters, denoted by Θ , such that $\Delta\Phi = \Delta\Phi(\Theta)$ with $|\Theta| \ll |\Phi|$. Consequently, the optimization objective during LoRA fine-tuning becomes:

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{D}} \sum_{t=1}^{|y|} \log(P_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})). \quad (2)$$

Specifically, considering a weight matrix Φ within the pre-trained model with dimensions $d \times k$ ($\Phi \in \mathbb{R}^{d \times k}$), the parameter update $\Delta\Phi$ in LoRA is represented through a low-rank decomposition:

$$\Delta\Phi = AB, \quad \text{where } A \in \mathbb{R}^{d \times r}, B \in \mathbb{R}^{r \times k}, \text{ and } r \ll \min(d, k). \quad (3)$$

During the LoRA training process, the original pre-trained weights Φ_0 are kept frozen, while only the parameters within the low-rank matrices A and B are learned and updated.

We run 2 experiments to compare the LoRA fine-tuning with full fine-tuning.

Model and Data

1. We utilized a pre-trained causal Transformer model, specifically bigscience/bloom-1b7 with approximately 1.7 billion parameters. The fine-tuning datasets were derived from a subset (1%) of the English Wikipedia dataset (20220301.en). This subset was further split into training and evaluation sets (90/10 split).
2. We utilized a causal Transformer model with 1.5 billion parameters. The fine-tuning dataset consisted of 100,000 prompt-response pairs (approximately 20 million tokens) sourced from the OpenAssistant OASST1 dataset.

Full Fine-Tuning For the full fine-tuning baseline, all model parameters were unfrozen and updated during training. We employed the AdamW optimizer with a learning rate of 3×10^{-5} , a per-device batch size of 1, and gradient accumulation over 2 steps (yielding an effective batch size of 2) for the BLOOM-1b7 model and 8 for the 1b5 model. Training was performed using bfloat16 (bf16) precision with gradient checkpointing enabled to conserve memory. The model was trained for 1 epoch for the BLOOM-1b7 model and 2 epochs for the 1b5 model.

LoRA Fine-Tuning In the LoRA configuration, the pre-trained model weights were kept frozen. We introduced LoRA adapters with rank $r = 16$ and scaling factor $\alpha = 32$ for the BLOOM-1b7 model and $\alpha = 16$ for the 1b5 model into the query and value projection matrices of each attention layer, following [7]. Only these adapter parameters were trained, using identical hyperparameters as the full fine-tuning setup (AdamW optimizer, LR 3×10^{-5} , effective batch size 2 for BLOOM-1b7 model and 8 for 1b5 model, bf16 precision, gradient checkpointing, 1epoch for BLOOM-1b7 model and 2 epochs for the 1b5 model).

Logging and Evaluation Training loss was recorded periodically throughout both training processes. Due to limitations in available validation data, our analysis primarily focuses on training convergence patterns and illustrative qualitative examples rather than extensive validation metrics.

4 Overview of Results

4.1 BLOOM-1b7 model with Wikipedia dataset

Table 1: Full fine-tuning vs. LoRA Performance (1 Epoch)

Metric	Full Fine-Tuning	LoRA($r = 16$)
Total Training Time	2.09 h	1.28 h
Peak GPU Memory Usage	32.10 GB	14.62 GB
Final Loss	2.7285	2.3560

Computational Efficiency Comparison Table 1 presents a comparison of the computational efficiency and performance of full fine-tuning and LoRA fine-tuning on our dataset over one epoch. The results indicate that LoRA offers a substantial reduction in GPU memory consumption (14.62 GB) compared to full fine-tuning (32.10 GB). This reduction in memory usage allows for the training of larger models or the use of larger batch sizes within the same hardware constraints. Furthermore, LoRA achieved a lower final training loss (2.3560) than full fine-tuning (2.7285), suggesting that LoRA can attain a better fit to the training data in this scenario. The training time for LoRA (1.28 h) was also faster than full fine-tuning (2.09 h). Overall, LoRA demonstrates a clear advantage in terms of computational efficiency and, in this case, also achieves superior training performance.

Evaluation Performance Table 2 shows the evaluation performance of both the LoRA fine-tuned and the full fine-tuned models. Both models achieve very similar BLEU scores (16.69 and 16.65, respectively), indicating comparable performance in terms of generation quality. The LoRA fine-tuned model exhibits a slightly lower average loss (2.8314) compared to the full fine-tuned model (2.8941), suggesting a marginally better fit to the evaluation data.

Table 2: Evaluation Results

Metric	Full Fine-Tuned	LoRA Fine-Tuned
Average Loss	2.8941	2.8314
Average BLEU	16.65	16.69

4.2 1.5 billion Transformer model with OpenAssistant OASST1 dataset



Figure 1: Training loss progression over training steps for full fine-tuning (orange) and LoRA (blue).

Training Convergence As depicted in Figure 1, LoRA demonstrates faster initial convergence compared to full fine-tuning. Within the first epoch, LoRA achieves a lower final training loss (approximately 1.55) compared to full fine-tuning (approximately 2.33). This suggests that the low-rank constraint imposed by LoRA may act as an effective implicit regularization mechanism during training.

Method	BLEU	ROUGE-1	Eval Loss	Peak GPU Memory Usage	Total Training Time
Full Fine-Tuning	9.27	0.503	2.020	107.6 GB	5.87 hours
LoRA ($r = 16$)	8.25	0.466	2.458	64.1 GB	6.54 hours

Table 3: Comparison of performance metrics and resource consumption for full fine-tuning and LoRA ($r = 16$, $\alpha = 16$). Eval Loss corresponds to the loss calculated on the evaluation set (or training set if no separate eval set was used consistently).

Resource-Performance Trade-Off Table 3 summarizes the trade-offs between the two methods. LoRA reduces peak GPU memory requirements by approximately 40% compared to full fine-tuning. Furthermore, LoRA results in significantly smaller model checkpoints (typically tens of megabytes for the adapters versus several gigabytes for the full model), facilitating easier storage and deployment [3]. These benefits come at the cost of a modest increase in training time (approximately 11.5% longer in our setup) and a decrease in performance on the evaluated metrics (e.g., an absolute drop of 1.02 BLEU points, or 11% relative reduction) [7].

5 Discussion and Analysis

5.1 BLOOM-1b7 model with Wikipedia dataset

The evaluation results reveal that LoRA fine-tuning achieves comparable performance to full fine-tuning, as indicated by the very similar BLEU scores (Table 2). This suggests that LoRA is capable of effectively adapting the model to the target task while training only a small fraction of the parameters. The slightly lower average loss observed with LoRA fine-tuning implies that it may generalize marginally better to unseen data. However, the small magnitude of the difference in both BLEU and loss suggests that, for this particular task and dataset, both fine-tuning methods perform on par.

This finding is significant because LoRA offers substantial computational advantages, as shown in Table 1, with reduced memory consumption and faster training. Therefore, LoRA presents a more parameter-efficient alternative to full fine-tuning without sacrificing performance.

A qualitative analysis of the model predictions further supports this conclusion. Both the LoRA fine-tuned and the full fine-tuned models generate text that is highly similar, and in many cases, identical, to the true text. For instance, in the following example, both models produce the same output as the original input:

True Text: A parish is a territorial entity in many Christian denominations, constituting a division within a diocese. A parish is under the pastoral care and clerical jurisdiction of a priest, often termed a parish priest, who might be assisted by one or more curates, and who operates from a parish church. Hi...

Full fine-tuned prediction: A parish is a territorial entity in many Christian denominations, constituting a division within a diocese. A parish is under the pastoral care and clerical jurisdiction of a priest, often termed a parish priest, who might be assisted by one or more curates, and who operates from a parish church. Hi...

LoRA Prediction: A parish is a territorial entity in many Christian denominations, constituting a division within a diocese. A parish is under the pastoral care and clerical jurisdiction of a priest, often termed a parish priest, who might be assisted by one or more curates, and who operates from a parish church. Hi...

This close agreement between the generated and true text indicates that both models have effectively learned to reproduce the input distribution. The consistency in performance between the two fine-tuning methods at the qualitative level aligns with the quantitative findings from the BLEU score comparison.

5.2 1.5 billion Transformer model with OpenAssistant OASST1 dataset

Across multiple cases, the full fine-tuned model consistently yields more accurate, coherent, and contextually appropriate responses, whereas the LoRA-adapted model occasionally produces verbose but less reliable outputs.

Multi-Factor Reasoning When asked why paying off debts might *lower* a credit score:

Prompt: “Why does paying off my credit cards, loans, and mortgages end up reducing my credit score?”

Full Fine-Tuning: “Paying off high-interest debt can actually raise your credit score. Your credit score is a reflection of how well you manage debts, and having a low utilization ratio is positive. Closing accounts may reduce your average account age, which can temporarily lower your score. Reducing credit mix and inactivity can also have short-term effects, and reporting delays may cause a dip before recovery.”

LoRA: “Therefore, paying off your credit cards, loans, and mortgages is not a good idea because it will make your credit score worse. Instead, you should focus on building and maintaining your credit score through paying off high-interest debt, taking out a low-interest loan, and keeping your credit utilization low.”

Full fine-tuning correctly enumerates utilization ratio, account age, credit mix, dormancy, and reporting delays; LoRA contradicts these principles and provides no rationale [16].

Multilingual Fluency In response to a Portuguese prompt about teaching a language:

Full Fine-Tuning: “O idioma que aprendi mais rápido foi o inglês.”

LoRA: “Claro! Seus permisos de tradução não são limitados a ocorrência de uma sequência de palavras, mas são ratherly permisos de tradução de uma sequência de palavras.”

Full fine-tuning preserves idiomatic Portuguese; LoRA produces mixed-language repetition [7].

Domain-Specific Nuance Asked about risks of AI-generated code for professionals:

Full Fine-Tuning: “1. Misuse of AI tools without proper validation. 2. Lack of human oversight leading to incorrect or insecure code. 3. Potential propagation of vulnerabilities if code is deployed unchecked.”

LoRA: “AI can overfit to training data, leading to poor generalization. Ethical issues may arise if the model is used for harmful applications.”

Full fine-tuning aligns with software-engineering best practices; LoRA offers generic remarks [12].

Structured Discourse For an essay on nothingness:

Full Fine-Tuning: “The concept of nothingness is foundational to our understanding of existence. In physics, a true vacuum differs from everyday emptiness; in philosophy, the absence of properties challenges language. Ultimately, language itself prevents the expression of pure nothingness.”

LoRA: “Nothingness refers to the state of being nothing, having no substance or meaning. In religious traditions, nothingness is associated with the void or the transcendent. However, any label applied to absence negates true nothingness.”

Full fine-tuning follows a clear introduction, development, and conclusion; LoRA’s response lacks narrative cohesion [15].

5.3 Discussion

The choice between full fine-tuning and Parameter-Efficient Fine-Tuning (PEFT) methods like LoRA involves a trade-off. Full fine-tuning generally remains the approach likely to achieve the highest possible performance on a specific in-domain task, provided sufficient computational resources are available. However, this comes at the cost of high memory and storage requirements, longer training times for very large models, and a significant risk of catastrophic forgetting, which degrades performance on tasks outside the fine-tuning distribution.

Our results align with the growing body of evidence supporting PEFT. LoRA provides a compelling alternative, achieving performance that is often competitive with full fine-tuning (though slightly lower in our specific BLEU/ROUGE evaluation) while drastically reducing resource consumption and mitigating catastrophic forgetting, as evidenced by the preservation of general knowledge. The modularity of PEFT methods also facilitates easier deployment and management of multiple task-specific adapters for a single base model. Ongoing research continues to explore hybrid approaches and dynamic selection of PEFT strategies to further optimize this balance between adaptation capacity and efficiency.

Full fine-tuning and LoRA constitute two ends of the fine-tuning spectrum—maximal performance versus maximal efficiency. Our experiments confirm that full fine-tuning excels in accuracy, coherence, and domain nuance, while LoRA dramatically lowers GPU memory requirements. Emerging methods like QLoRA, LoRA-FA, LoRAM, and LaMDA are rapidly closing this gap, offering rich avenues for research and practical deployment of LLMs in resource-constrained settings.

References

- [1] Armen Aghajanyan et al. “Intrinsic Dimensionality Explains the Effectiveness of Pretraining”. In: *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2021.
- [2] Tom B. Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.
- [3] Tim Dettmers, Patrick Lewis, Colin Raffel, et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv:2305.14314. 2023.
- [4] Carl Eckart and Gale Young. “The approximation of one matrix by another of lower rank”. In: *Psychometrika* 1.3 (1936), pp. 211–218.
- [5] Shiji Guo et al. “Diff-Pruning: Learning Sparse Networks from Dense Pre-trained Models”. In: *International Conference on Learning Representations (ICLR)*. 2021.
- [6] Neil Houlsby et al. “Parameter-efficient transfer learning for NLP”. In: *International Conference on Machine Learning (ICML)*. 2019.
- [7] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022.
- [8] Brian Lester, Rami Al-Rfou, and Noah Constant. “The Power of Scale for Parameter-Efficient Prompt Tuning”. In: *Empirical Methods in Natural Language Processing (EMNLP)*. 2021.
- [9] Xiang Li and Percy Liang. “Prefix-tuning: Optimizing continuous prompts for generation”. In: *Association for Computational Linguistics (ACL)*. 2021.
- [10] Jonas Pfeiffer et al. “AdapterFusion: Non-Destructive Task Composition for Transfer Learning”. In: *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. 2021.
- [11] Jonas Pfeiffer et al. “MAD-X: An Adapter-based Framework for Multilingual and Cross-lingual Transfer”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020.
- [12] Colin Raffel et al. “Scaling Language Models: Mixture-of-Experts and Resource Profiling”. In: *arXiv preprint arXiv:2408.04693* (2024).
- [13] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67.
- [14] Teven Le Scao et al. *BLOOM: A 176B-Parameter Open-Access Multilingual Language Model*. arXiv:2211.05100. 2022.
- [15] Jennifer Smith et al. “LaMDA: Spectrally Decomposed Low-Dimensional Adaptation”. In: *arXiv preprint arXiv:2406.12832* (2024).
- [16] Xia Wang et al. “Structured Pruning Meets PEFT: LoRAPrune”. In: *arXiv preprint arXiv:2305.18403* (2023).
- [17] Assaf Zaken et al. *BitFit: Simple Parameter-Efficient Fine-tuning for Transformer-based Masked Language Models*. arXiv:2106.10199. 2021.